Chapter 5: Temporal-Difference Learning

Hoda Zamani

PhD in Artificial Intelligence and Robotics

GitHub: hodazamani.github.io

Hoda Zamani - Google Scholar

Primary Textbook

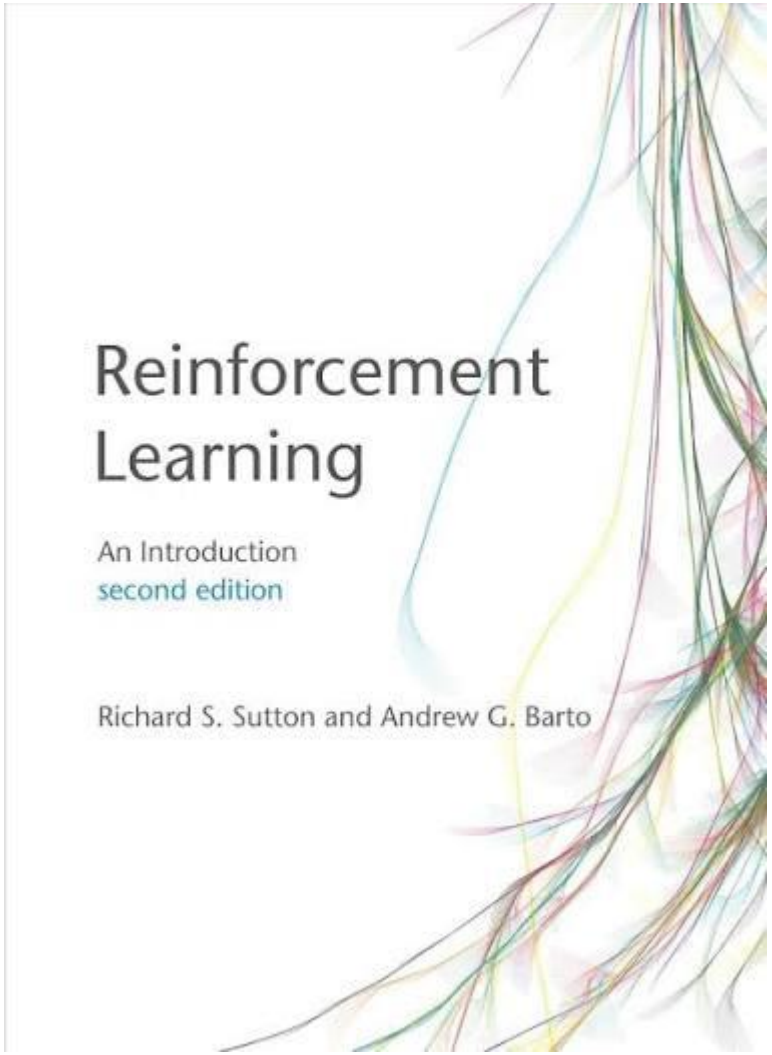
Reinforcement Learning: An Introduction, Second edition, in progress
Richard S. Sutton and Andrew G. Barto, Download from: SuttonBartoIPRLBook2ndEd.pdf

Recommended References

- Szepesvári, C. (2010). Algorithms for Reinforcement Learning. Morgan & Claypool Publishers.
- Bertsekas, D. P. (2019). Reinforcement Learning and Optimal Control. Athena Scientific.
- Puterman, M. L. (2014). Markov Decision Processes: Discrete Stochastic Dynamic Programming. Wiley.

Additional Course Materials

- Instructor-provided lecture notes, slides, and supplementary reading materials.
- Programming assignments and practical exercises using Python and relevant machine learning libraries.
- Selected research papers and technical articles on reinforcement learning and decision-making.
- Computational demonstrations, datasets, and examples provided throughout the course.



You can find:


Course announcements & communication


Slides & weekly schedule


Textbook & reading materials


TA


Course policies

Here: https://hodazamani.github.io/Courses/

Assessment Component	Weight
Weekly Assignments, Quiz & Programming Exercises	20%
Midterm Examination	30%
Reinforcement Learning Project/Presentation	20%
Final Examination	30%
Total	**100%**

CONTENTS

- **Temporal-Difference (TD) Learning**
- **TD Prediction**
- **Monte Carlo Control**
- **Advantages of TD Prediction Methods**
- **Optimality of TD(0)**
- **SARSA: On-Policy TD Control**
- **Summary**

Temporal-Difference (TD) Learning

- **TD learning** is a fundamental idea in Reinforcement Learning that combines Monte Carlo (MC) and Dynamic Programming (DP) methods.
- Like **Monte Carlo**, TD learns directly from experience without requiring a model of the environment.
- Like **Dynamic Programming**, TD uses previously learned estimates to update current estimates. This is called **bootstrapping**.
- Unlike Monte Carlo, TD can update value estimates after each step without waiting until the end of an episode.

TD vs. MC vs. DP

Feature	Monte Carlo	Dynamic Programming	TD Learning
Learns from experience	Yes	Not necessarily	Yes
Requires an environment model	No	Yes	No
Bootstrapping	No	Yes	Yes
Must wait until episode ends	Usually yes	No	No

Prediction and Control

- **Prediction (Policy Evaluation):** Estimate the value function v_π for a given policy π.
- **Control:** Find an optimal policy that maximizes expected cumulative reward.
- **Generalized Policy Iteration (GPI):** Combines policy evaluation and policy improvement to find a better policy iteratively.

- **Temporal-Difference (TD) learning** estimates the state-value function $v_\pi(s)$ for a given policy π.
- Both TD and Monte Carlo (MC) methods learn from experience.
- The goal is to estimate the expected return from each state under policy π.
- The main difference is when learning occurs:
  - **Monte Carlo:** waits until the episode ends.
  - **TD:** updates after each time step using the next reward and the estimated value of the next state.

Monte Carlo (MC)

$$V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)]$$

- Uses the actual return G_t as the learning target.
- Does not bootstrap.
- Must wait until the return is known, usually at the end of the episode.

Temporal-Difference (TD(0))

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

- Uses the immediate reward and the estimated value of the next state.
- Updates after one time step.
- Uses bootstrapping.

TD(0) Update Rule

The most important equation in this section is:

$$V(S_t) \leftarrow V(S_t) + \alpha\delta_t$$

where the **TD error** is

$$\delta_t = \underbrace{R_{t+1} + \gamma V(S_{t+1})}_{\text{TD target}} - V(S_t)$$

Symbol	Meaning
$V(S_t)$	Current estimate of the state value
α	Step-size or learning-rate parameter
R_{t+1}	Immediate reward
γ	Discount factor
$V(S_{t+1})$	Estimated value of the next state
δ_t	TD error: difference between target and current estimate

- **TD(0) is a one-step, model-free prediction method.**
- It estimates v_π, the value function of a given policy.
- MC learns from actual returns; TD learns from one-step sampled targets.
- TD combines Monte Carlo sampling with DP-style bootstrapping.
- TD can learn before an episode ends.

Why Is TD Learning Called Bootstrapping?

TD learning uses its existing estimate $V(S_{t+1})$ to improve the estimate $V(S_t)$.

The Bellman expectation equation is:

$$v_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s]$$

- The true value $v_\pi(S_{t+1})$ is generally unknown.
- TD substitutes the current estimate $V(S_{t+1})$.
- The estimate is therefore updated using another estimate.

This process is called **bootstrapping**.

Remember: TD combines the sampling of Monte Carlo methods with the bootstrapping of Dynamic Programming (DP).

Comparing MC, TD, and DP

Feature	Monte Carlo	TD(0)	Dynamic Programming
Learns from experience	Yes	Yes	Uses an environment model
Target	Actual return G_t	$R_{t+1} + \gamma V(S_{t+1})$	Expected one-step return
Bootstrapping	No	Yes	Yes
Requires a complete episode	Usually yes	No	No
Update timing	After return is known	Each time step	From model-based backups

Key takeaway: TD does not need a complete episode or a known model of the environment.

Tabular TD(0) for Policy Evaluation

1. Initialize $V(s)$ arbitrarily for all states, for example $V(s) = 0$.
2. Initialize the starting state S.
3. Choose action A according to policy π.
4. Take action A and observe reward R and next state S'.
5. Update the current state's value:

$$V(S) \leftarrow V(S) + \alpha[R + \gamma V(S') - V(S)]$$

6. Set $S \leftarrow S'$ and repeat until the state is terminal.
7. Repeat for each episode.

Sample Backups

TD and Monte Carlo use **sample backups**.

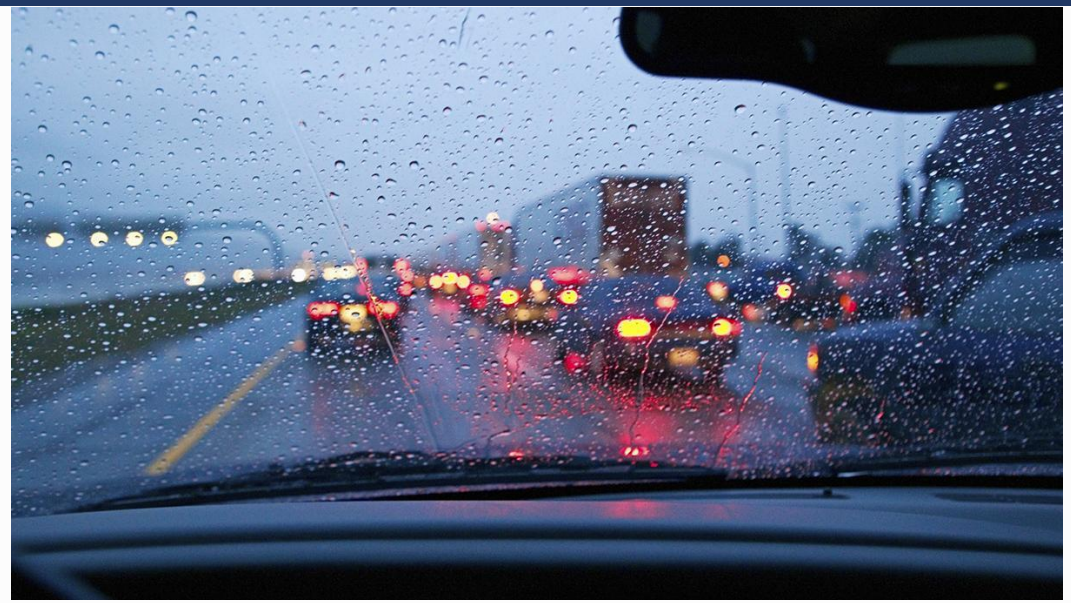
- A sample backup uses one observed transition to update a state-value estimate.
- It looks at the reward and a sampled successor state.
- It does not require the complete probability distribution of all possible next states.

In contrast, DP uses **full backups**, which calculate expectations over all possible successor states using the environment model.

Example — Driving Home

Suppose you estimate that driving home will take 30 minutes.

- After reaching your car, rain increases your estimate to 40 minutes total.
- After exiting the highway, you estimate 35 minutes total.
- A slow truck delays you, and you finally arrive home after 43 minutes.



The true return from each state is the actual remaining travel time.

Situation	Predicted total time	Actual total time
Leaving office	30 min	43 min
Reaching car	40 min	43 min
Exiting highway	35 min	43 min
Behind truck	40 min	43 min
Entering home street	43 min	43 min

Here, $\gamma = 1$, so the return equals the actual remaining travel time.

Monte Carlo approach

- Waits until arriving home to know the actual return.
- Updates predictions using the difference between actual return and current prediction.
- For example, at highway exit, the prediction is 15 minutes remaining, but the actual time remaining is 23 minutes.
- The prediction error is $23 - 15 = 8$ minutes.

With $\alpha = 0.5$:

$$V_{\text{new}} = 15 + 0.5(23 - 15) = 19$$

TD approach

- Updates estimates immediately using the next state's prediction.
- Does not need to wait until arriving home.
- Each prediction is moved toward the target implied by the next observed reward and the next state's current estimate.

For example, if the first estimate is 30 minutes and the next state suggests a total journey time of 50 minutes, TD can adjust the first estimate immediately.

Advantages of TD Prediction Methods

- TD methods learn by using current estimates to update other estimates. This process is called **bootstrapping**.
- Unlike Dynamic Programming (DP), TD methods do not require a model of the environment.
- Unlike Monte Carlo (MC), TD methods do not need to wait until the end of an episode.
- TD methods learn incrementally from each observed transition.
- They are useful for long episodes and continuing tasks.

Advantages of TD over MC and DP

Compared with Dynamic Programming:

- TD does not require the environment's reward and next-state probability distributions.
- It learns directly from experience without a complete model.

Compared with Monte Carlo:

- TD updates after one time step, whereas MC usually waits until the episode ends.
- TD can learn from each transition, even when subsequent actions differ.
- This makes TD useful when episodes are long or when tasks have no terminal states.

Convergence of TD Methods

- TD methods can converge to the correct value function despite using estimates to update estimates.
- For a fixed policy, tabular TD(0) converges to (v_π) under appropriate conditions.
- With a sufficiently small constant step size, convergence occurs in the mean.
- With a decreasing step size satisfying standard stochastic approximation conditions, convergence occurs with probability 1.
- Some convergence results also apply to linear function approximation.

Key point: TD learning is theoretically supported, provided the required conditions are satisfied.

Which Method Learns Faster?

- Both TD and Monte Carlo can converge to the correct predictions.
- An important question is which method learns faster and makes better use of limited experience.
- There is no general mathematical proof that one method always converges faster.
- In practice, TD methods have often performed better than constant alpha Monte Carlo methods on stochastic tasks.



Figure 6.5: A small Markov process for generating random walks.

- The process contains five states: A, B, C, D, and E.
- Each episode starts in the center state, C.
- At each step, the process moves one state to the left or right with equal probability.
- The episode terminates upon reaching A or E.
- Reaching E produces a reward of +1; all other rewards are zero.
- The true value of each state is the probability of reaching the right terminal state.

State Values Learned by TD(0)

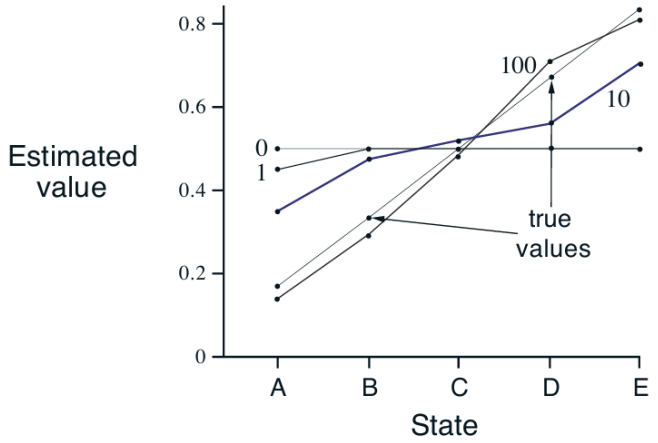


Figure 6.6: Values learned by TD(0) after various numbers of episodes. The final estimate is about as close as the estimates ever get to the true values. With a constant step-size parameter ($\alpha = 0.1$ in this example), the values fluctuate indefinitely in response to the outcomes of the most recent episodes.

- Initially, all state values are set to 0.5.
- As more episodes are experienced, TD(0) updates the estimates toward the true values.
- The learned values approach the true state values.
- With a constant step size, such as (alpha=0.1), the estimates continue to fluctuate because they respond to recent outcomes.

Key point: TD(0) progressively improves its predictions through repeated experience.

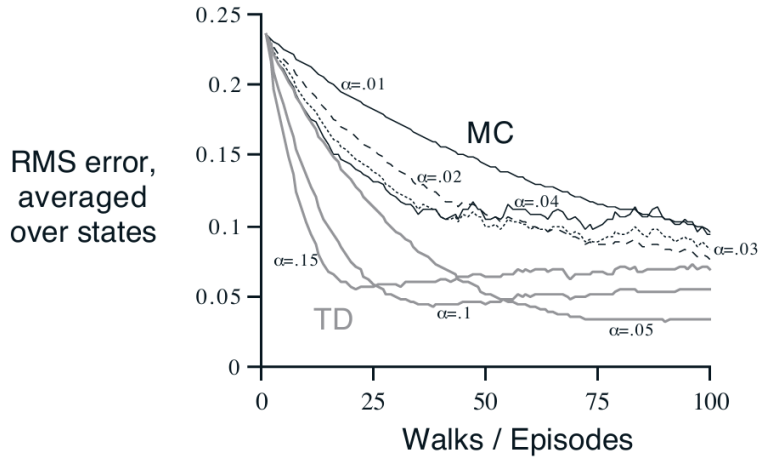


Figure 6.7: Learning curves for TD(0) and constant-α MC methods, for various values of α, on the prediction problem for the random walk. The performance measure shown is the root mean-squared (RMS) error between the value function learned and the true value function, averaged over the five states. These data are averages over 100 different sequences of episodes.

- The figure compares TD(0) and constant-(\alpha) MC across different learning rates.
- Performance is measured using root mean-squared (RMS) error between estimated and true state values.
- The errors are averaged across the five states and 100 sequences of episodes.
- In this experiment, TD(0) consistently achieves lower prediction error than constant alpha MC over the tested episodes and learning rates.

What Is Batch Updating?

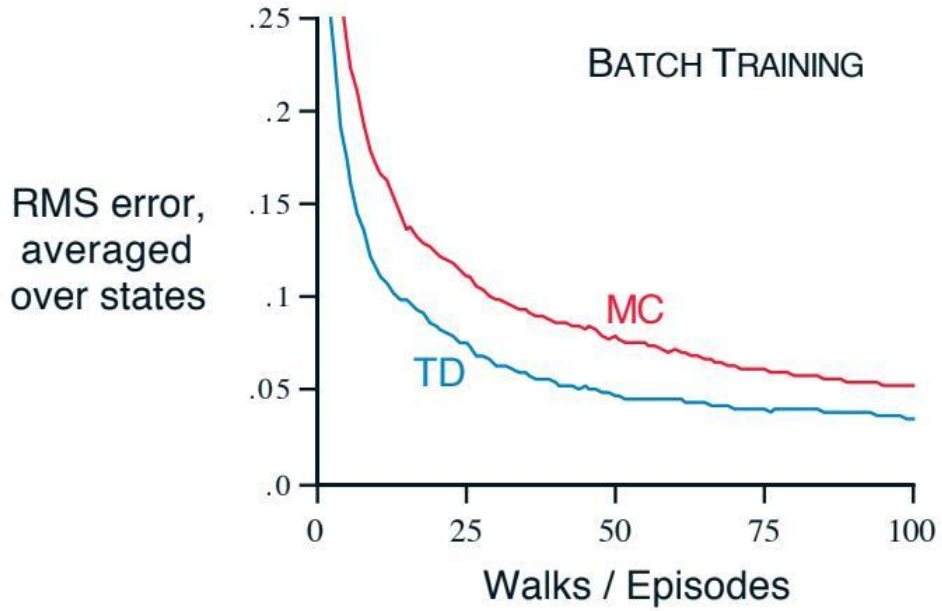
- Collect a fixed batch of experience, such as 10 episodes or 100 time steps.
- Process the entire batch repeatedly.
- Calculate TD or MC updates for every time step.
- Apply the sum of updates only after processing the full batch.
- Repeat until the value estimates converge.

Batch TD(0) vs. Batch Monte Carlo

Batch TD(0)	Batch Monte Carlo
Uses bootstrapping	Uses actual observed returns
Learns from transitions and estimated next-state values	Learns from complete returns
Converges to the certainty-equivalence estimate	Converges to sample-average returns
Not necessarily the lowest training-set error	Minimizes squared error on training returns

- With a sufficiently small step size, each batch method converges deterministically to its own solution.

Random Walk Under Batch Updating



- Apply batch TD(0) and constant-step-size Monte Carlo to the random-walk task.
- After each episode, replay all episodes observed so far.
- Repeat until the value function converges.
- Measure RMS error across the five nonterminal states and 100 independent experiments.
- **Result:** batch TD(0) consistently performs better than batch Monte Carlo in the plotted experiment.

Example: You Are the Predictor

The observed episodes are:

Plain text

```
A, 0, B, 0
B, 1
B, 1
B, 1
B, 1
B, 1
B, 1
B, 0
```

- State B is observed eight times.
- Six visits to B produce return 1.
- Two visits to B produce return 0.
- Therefore, $V(B) = 6/8 = 0.75$.

Two Plausible Estimates for State A

Monte Carlo

$$V(A) = 0$$

Only observed return from A was zero.

TD(0)

$$V(A) = 0.75$$

A transitions to B with reward zero, so $V(A) = 0 + V(B)$.

- Build a maximum-likelihood model from observed transitions and rewards.
- Estimate transition probabilities from observed frequencies.
- Estimate expected rewards from observed rewards.
- Calculate the value function that would be correct for this estimated model.
- Batch TD(0) converges to this **certainty-equivalence estimate**.

Computational Cost of Certainty Equivalence

- For N states, estimating the transition model may require $O(N^2)$ memory.
- Conventional solution of the value equations may require $O(N^3)$ computation.
- Tabular TD stores only state values: $O(N)$ memory.
- TD repeatedly processes experience without explicitly constructing the full model.

What Makes TD Different?

- TD learns by bootstrapping from the next state's estimated value.
- Batch Monte Carlo minimizes squared error on observed returns.
- Batch TD converges to the certainty-equivalence estimate.
- TD may generalize better to future experience.
- Online TD often learns quickly, but its relative efficiency depends on the task.

From TD Prediction to TD Control

- **Prediction:** estimate the value of each state, $V(s)$.
- **Control:** learn which actions are best in each state.
- Estimate action values, $Q(s, a)$.
- Follow generalized policy iteration:
  1. Evaluate the current policy.
  2. Improve the policy to favor better actions.
- Sarsa is an **on-policy TD control** algorithm.

State Values vs. Action Values

State value	Action value
$V^\pi(s)$	$Q^\pi(s, a)$
Expected return from state s under policy π.	Expected return after taking action a in state s, then following policy π.

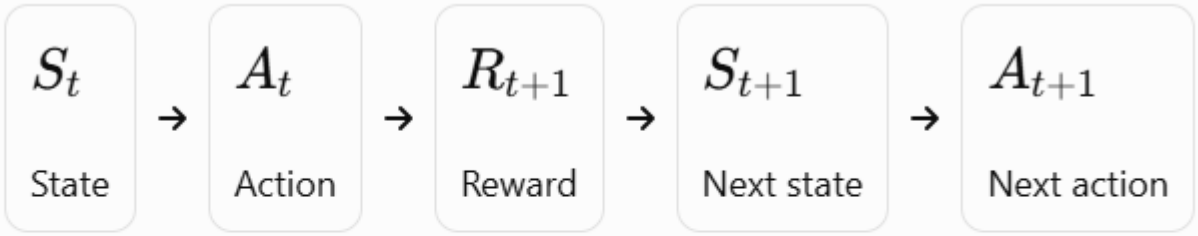
- Sarsa learns values for state–action pairs.
- A transition includes the next action, not just the next state.
- The next action is selected using the current behavior policy.

The Sarsa Update Rule

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\big[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\big]$$

- S_t: current state
- A_t: current action
- R_{t+1}: reward received
- S_{t+1}: next state
- A_{t+1}: next action selected
- α: step size; γ: discount factor

Why Is It Called SARSA?

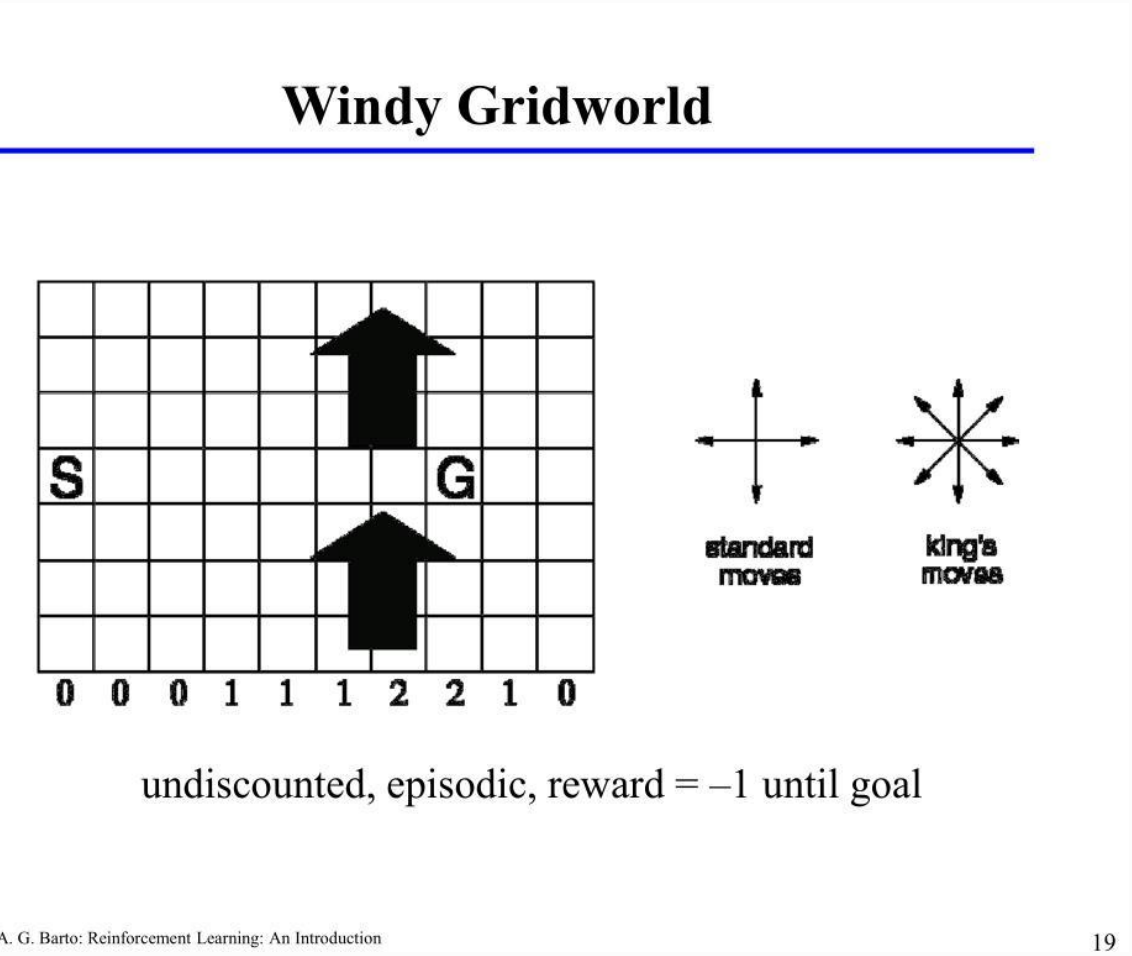


- The update uses the quintuple $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$.
- If the next state is terminal, the next action value is defined as zero.

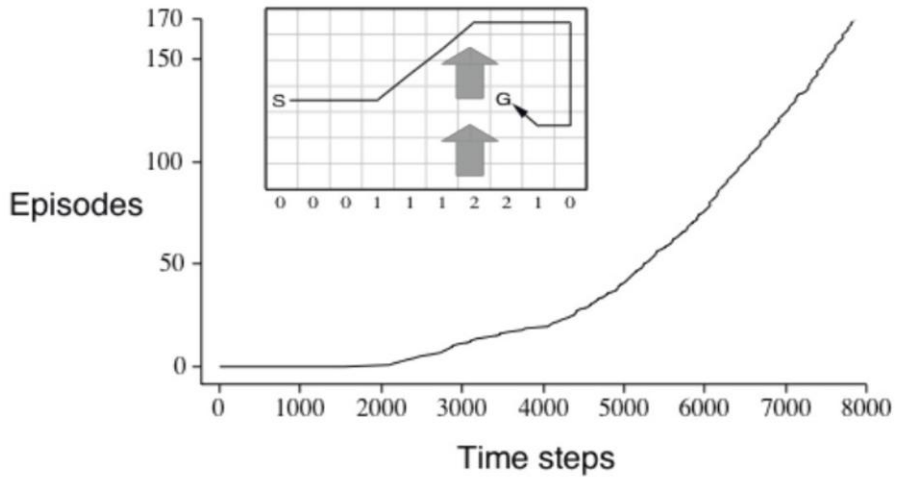
1. Initialize $Q(s, a)$ arbitrarily for every state–action pair.
2. Start an episode and observe the initial state S.
3. Choose action A using a policy derived from Q, such as ϵ-greedy.
4. Take action A, observe reward R and next state S'.
5. Choose next action A' using the same behavior policy.
6. Update $Q(S, A)$ using the Sarsa equation.
7. Set $S \leftarrow S'$ and $A \leftarrow A'$; repeat until terminal.

Example: Windy Gridworld

- The agent must travel from start S to goal G.
- Available actions: up, down, left, right.
- A location-dependent wind shifts the agent upward.
- Each nonterminal move gives reward -1.
- Example parameters:
  - $\epsilon = 0.1$
  - $\alpha = 0.5$
  - $Q(s, a) = 0$ initially



Results of Sarsa on the Windy Gridworld



- The x-axis measures time steps.
- The y-axis measures the number of episodes completed.
- The slope increases as the agent learns to reach the goal more quickly.
- The learned greedy policy becomes optimal well before 8,000 time steps.
- Continued exploration keeps the average episode length around 17 steps; the minimum is 15.

SARSA: Main Takeaways

- Sarsa is an on-policy TD control algorithm.
- It learns action values $Q(s, a)$, not just state values.
- It updates after each transition using the next action selected by the current policy.
- Epsilon-greedy behavior balances exploration and exploitation.
- Under suitable conditions, Sarsa can converge to an optimal policy.

Question?